BANNARI AMMAN INSTITUTE OF TECHNOLOGY
An Autonomous Institution Affiliated to Anna University - Chennai • Approved by AICTE • Accredited by NAAC with 'A+' Grade
SATHYAMANGALAM - 638401 ERODE DISTRICT TAMILNADU INDIA
Ph: 04295-226000/221289 Fax: 04295-226666 Email: stayahead@bitsathy.ac.in Web: www.bitsathy.ac.in

Academic Year 2024 - 25

S8 PROJECT WORK II

First Review

PROJECT TITLE

SmartGig: A Freelancing Platform for Modern Workforce Management

BIP PROJECT ID
2026IN492

CATEGORY
INTERNAL

BATCH MEMBERS

Raajasri K (7376222IT228)

Dikshana K (7376222IT134)

Deepak Kumar V (7376222CT107)

GUIDE

Mrs. Sandhiya B

Assistant Professor

Department of Information Technology

LITERATURE SURVEY

Sl.No.	Journal Paper Title with Author	Works carried out (with details of Methods/ Materials/ Software/ Algorithms / fabrication / techniques/ components used)	Information gathered relevant to your project
1	**Alvarez de la Vega, J.C., Cecchinato, M.E., & Rooksby, J. (2022). Design Opportunities for Freelancing Platforms: Online Freelancers' Views on a Worker-Centred Design Fiction.** *2022 Symposium on Human-Computer Interaction for Work, ACM.* Link	Developed and evaluated 'Freelance Grow,' a design fiction portraying a freelancing platform that prioritizes freelancers' professional development and peer support.	Provides insights into design approaches that support freelancers' autonomy, professional growth, and peer interactions, which can inform the development of a freelancer-centric platform.
2	**J.C., Cecchinato, M.E., & Rooksby, J. (2022). Design Opportunities for Freelancing Platforms: Online Freelancers' Views on a Worker-Centred Design Fiction.** *2022 Symposium on Human-Computer Interaction for Work, ACM.* Link	Investigated freelancers' experiences with gig economy platforms, focusing on issues of control, autonomy, and platform governance.	Highlights the challenges freelancers face regarding control and autonomy on existing platforms, suggesting the need for designs that offer greater freelancer empowerment.

LITERATURE SURVEY

Sl.No.	Journal Paper Title with Author	Works carried out (with details of Methods/ Materials/ Software/ Algorithms / fabrication / techniques/ components used)	Information gathered relevant to your project
3	**Alvarez de la Vega, J.C., Cecchinato, M.E., Rooksby, J., & Newbold, J. (2023). Understanding Platform Mediated Work-Life: A Diary Study with Gig Economy Freelancers.** *Proceedings of the ACM on Human-Computer Interaction, 7(CSCW1), 1–32.*	Conducted a diary study to explore how platform features impact freelancers' work-life balance and practices.	Provides empirical data on how platform design affects freelancers' daily routines and work-life balance, informing the need for features that support flexibility and well-being.
4	**Anwar, M.A., & Graham, M. (2020). Between a Rock and a Hard Place: Freedom, Flexibility, Precarity, and Vulnerability in the Gig Economy in Africa.** *Competition & Change, 24(5), 486–505.*	Analyzed the dual nature of gig work, offering both freedom and flexibility, but also precarity and vulnerability, particularly in the African context.	Highlights the precarious aspects of gig work, emphasizing the importance of designing platforms that provide security and support for freelancers.

AIM & OBJECTIVES OF THE PROJECT
(Problem Statement)

AIM:

To design and develop a smart, AI-enabled freelancing platform that efficiently connects freelancers and employers while ensuring secure payments, transparency, and intelligent job recommendations.

OBJECTIVES:

1. Milestone-Based Escrow Payments
2. Skill Validation Through Work Samples
3. Freelancer Workload Protection
4. Proof-of-Work Based Hiring
5. Multi-Dimensional Reputation System
6. Multi-Layer Violation & Ban System

SCOPE OF THE PROJECT

1. **Design and implement a role-based full-stack web application** with secure authentication and authorization mechanisms.
2. **Define and enforce role-specific workflows** for Clients, Freelancers, and Admins with controlled access to system features.
3. **Implement server-side business logic** for job posting, proposal submission, and contract creation.
4. **Simulate an escrow-based payment mechanism** using ledger entries and transaction state management.
5. **Provide dashboard interfaces** for tracking jobs, proposals, contracts, and earnings.
6. **Implement audit logging and transparency mechanisms** for financial and system-critical actions.
7. **Ensure data validation, error handling, and graceful failure**, while structuring the system for future extensions such as milestones, real payments, chat, and video calls.

NEED FOR THE CURRENT STUDY

1. Existing freelancing platforms are complex and opaque, making it difficult to understand internal workflows such as hiring, escrow handling, and dispute resolution.
2. There is a need to study secure role-based system design that clearly separates client, freelancer, and administrator responsibilities.
3. Real-world freelancing systems require strong backend workflows and transaction state management, which this study aims to understand and implement.
4. Understanding escrow-based payment models is essential for building trust in digital marketplaces, yet rarely explored in student projects.
5. Modern web systems must be resilient to failures; this study emphasizes validation, error handling, and graceful degradation.
6. The project provides a scalable architectural foundation that can be extended to real payments, chat systems, and milestone-based workflows in the future.

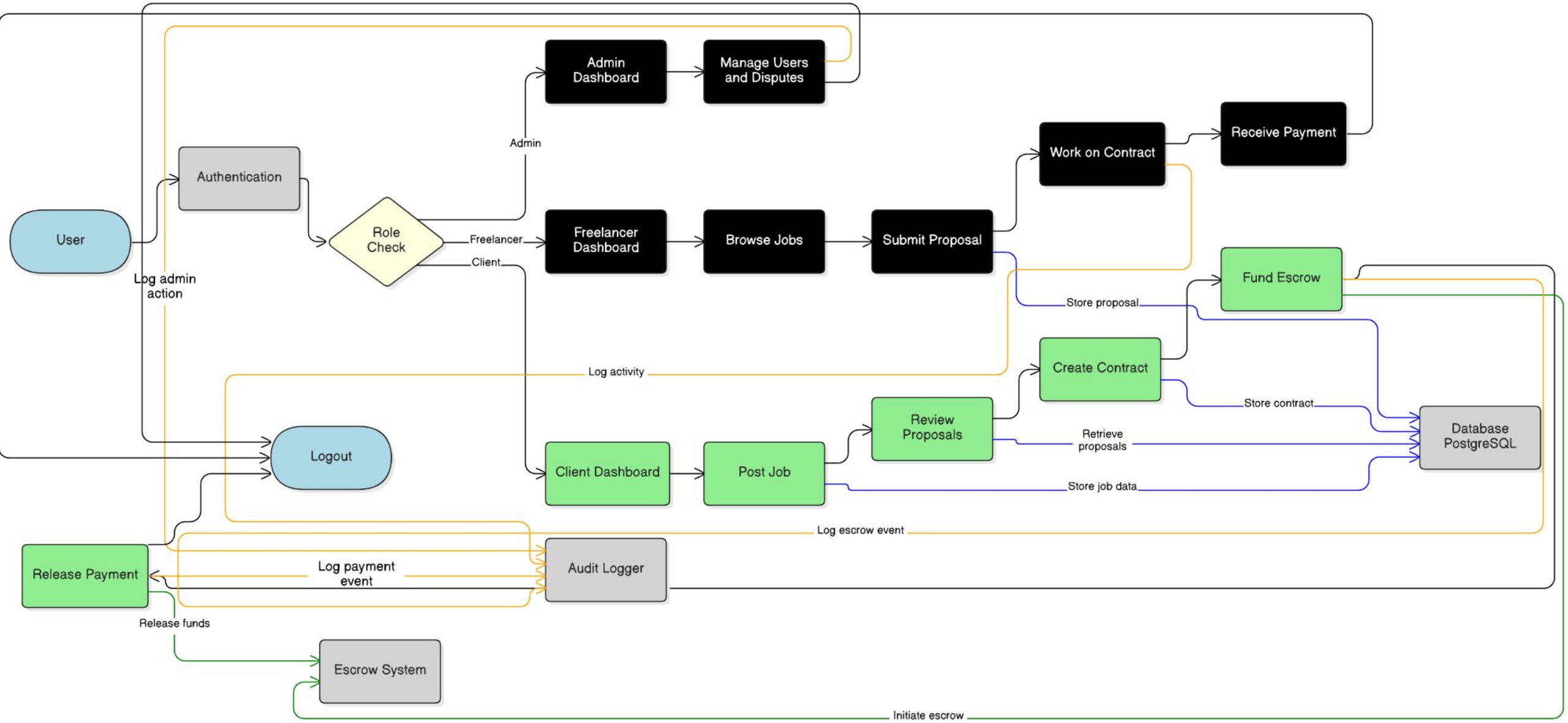
FEASIBILITY ANALYSIS

1. **Technical Feasibility:** Built using proven technologies (Next.js, PostgreSQL, Prisma)
2. **Implementation Feasibility:** Modular architecture allows incremental development
3. **Database Feasibility:** Relational schema supports users, jobs, proposals, and transactions
4. **Authentication Feasibility:** Secure role-based access using established auth patterns
5. **Payment Feasibility:** Escrow logic simulated through ledger and transaction states
6. **Scalability Feasibility:** Design supports future additions like milestones and real payments
7. **Operational Feasibility:** System runs on standard development environments
8. **Testing Feasibility:** Features verified through manual testing and E2E workflows
9. **Maintenance Feasibility:** Clean separation of frontend, backend, and services
10. **Risk Feasibility:** Mocked integrations reduce dependency on external services
11. **Time Feasibility:** Core MVP achievable within academic project timelines
12. **Cost Feasibility:** Uses open-source tools, minimizing infrastructure and licensing costs

FEASIBILITY ANALYSIS

13. **Deployment Feasibility:** Application can be deployed using standard cloud or local server setups
14. **Security Feasibility:** Input validation, access control, and audit logging reduce misuse risks
15. **Extensibility Feasibility:** Feature boundaries allow future modules to be added without redesign
16. **Operational Feasibility:** System workflows align with real freelancing platform operations
17. **Maintainability Feasibility:** Modular folder structure allows easy debugging and future maintenance
18. **Testing Feasibility:** Core flows can be verified through controlled demo data and test scenarios
19. **Learning Feasibility:** Technologies used are well-documented and suitable for academic learning
20. **Integration Feasibility:** Backend services are structured to integrate third-party tools when required
21. **Performance Feasibility:** Designed for moderate user loads typical of an academic project scope
22. **Documentation Feasibility:** Clear code structure and audit logs support effective project documentation

PROPOSED METHODOLOGY (Flow Chart)



PROPOSED METHODOLOGY (Gantt Chart)

TASK	WEEK 1	WEEK 2	WEEK 3	WEEK 4	WEEK 5	WEEK 6	WEEK 7	WEEK 8	WEEK 9	WEEK 10	WEEK 11	WEEK 12
Step 1: Requirement Gathering & analysis												
Step 2: System Architecture & Database Design												
Step 3: UI/UX Design & Prototyping												
Step 4: Frontend Development & Component Library												
Step 5: Authentication & User Management												
Step 6: Profile & Portfolio Management												
Step 7: Key Marketplace Features												
Step 8: Proposal & Bidding System												
Step 9: Real-Time Chat System												
Step 10: Contract & Milestone Management												
Step 11: Testing & Optimization												
Step 12: Final Deployment & Documentation												

CHOICE OF COMPONENTS / MODULES / METHODS/TECHNIQUES EQUIPMENT USED FOR PROJECT DEVELOPMENT

Frontend Technologies

Technology	Purpose
Next.js (React 19)	Server-side rendering, API routes, file-based routing, optimized performance
Tailwind CSS 3.4	Utility-first CSS, rapid UI development, consistent design system
Lucide React	Consistent, lightweight icon library
Sonner	Toast notifications for user feedback
Framer Motion	Smooth page transitions and micro-animations
Three.js + React Three Fiber	Render interactive, performance-optimized 3D and particle-based visuals on the landing page using WebGL.
Custom UI Components	Glassmorphism design system (GlassCard, GlassButton, GlassInput)

CHOICE OF COMPONENTS / MODULES / METHODS/TECHNIQUES EQUIPMENT USED FOR PROJECT DEVELOPMENT

Backend Technologies

Technology	Role	Purpose
Node.js 20 LTS	Runtime	JavaScript ecosystem, non-blocking I/O, large package ecosystem
PostgreSQL 18	Primary database	ACID compliance, complex queries, JSON support, reliability
Prisma ORM 5.10	Database Access Layer	Type-safe queries, auto-generated client, easy migrations
NextAuth.js v5 (Auth.js)	Authentication	Email/Password, Google OAuth, GitHub OAuth

Security Components

Technology	Role
FingerprintJS	Source code version control
bcrypt	Password Hashing
Zod Validation	Input validation

CHOICE OF COMPONENTS / MODULES / METHODS/TECHNIQUES EQUIPMENT USED FOR PROJECT DEVELOPMENT

Real-Time & Communication

Technology	Purpose
WebSocket: Socket.IO 4.8	Reliable real-time communication, auto-reconnection, room management
Video Conferencing: Jitsi Meet SDK	Open-source, no per-minute costs, easy integration

Version Control & Deployment

Tool	Implementation
Git	Source code version control
GitHub	Remote repository hosting
Vercel	Production deployment (planned)

CHOICE OF COMPONENTS / MODULES / METHODS/TECHNIQUES EQUIPMENT USED FOR PROJECT DEVELOPMENT

Development & Supporting Tools

Tool	Purpose
TypeScript 5.3	Static typing, better IDE support, fewer runtime errors
ESLint	Code quality enforcement
Prettier	Consistent code formatting
VS Code	Primary IDE with extensions

Testing Framework

Tool	Purpose
Playwright	End-to-end browser testing
Jest	Unit testing (planned)

DESIGN(S) (HARDWARE / SOFTWARE ARCHITECTURE)

1. Overall Architectural Approach

The system is designed as a **Modular Monolithic Web Application** following a **Three-Tier Architecture**.

- **Architecture Type: Modular Monolith**
  1. Frontend and backend are implemented within a **single unified codebase**.
  2. The system is logically divided into modules such as **Authentication, Jobs, Proposals, Contracts, Payments, and Chat**.
  3. This approach simplifies development and deployment while keeping the system **future-ready** for migration into microservices if required.

2. Software Architecture (Three-Tier Design)

A. Presentation Layer (Client Layer)

This layer is responsible only for **display and interaction**, while all critical logic is handled server-side.

Aspect	Description
Technology	React-based UI rendered via modern web browsers
Responsibilities	User interface rendering, form handling, dashboard views, and user interaction
Design Approach	Component-based design with reusable UI components
User Roles	Separate dashboard views for Clients, Freelancers, and Admins

DESIGN(S) (HARDWARE / SOFTWARE ARCHITECTURE)

B. Application Layer (Server / Business Logic Layer)

Aspect	Description
Technology	Server-side application logic using Node.js runtime
Responsibilities	Authentication, authorization, business rules, escrow simulation, workflow management
Security	Role-based access control enforced via middleware
Validation	Strict input validation before processing requests

DESIGN(S) (HARDWARE / SOFTWARE ARCHITECTURE)

C. Data Layer (Storage Layer)

This layer ensures **data integrity, persistence, and traceability**.

Aspect	Description
Database	Relational database system
Stored Data	Users, profiles, jobs, proposals, contracts, transactions, messages, audit logs
Access Pattern	ORM-based data access ensuring consistency and type safety
Reliability	ACID-compliant transactions for financial operations

4. Hardware Architecture (Three-Tier Design)

Server-Side Hardware

Component	Role
Application Server	Executes frontend rendering and backend logic
Database Server	Stores persistent system data
Network	Secure internet-based communication (HTTPS)

Client-Side Hardware

Component	Role
User Device	Laptop, desktop, tablet, or mobile device
Access Method	Modern web browser
Design Support	Responsive UI adapts to different screen sizes

INDIVIDUAL CONTRIBUTIONS TO THE WORK

Batch Member 1 : RAAJASRI K (7376222IT228)

1. Designed the overall user interface and layout of the application
2. Developed responsive dashboards and pages using modern frontend frameworks
3. Implemented visual components, navigation flow, and user interaction design
4. Ensured consistency in styling and usability across all user roles

INDIVIDUAL CONTRIBUTIONS TO THE WORK

Batch Member 2 : DIKSHANA K (7376222IT134)

1. Integrated frontend and backend modules into a cohesive system
2. Handled error handling, validation, and system stability improvements
3. Conducted functional testing and end-to-end verification of workflows
4. Prepared project documentation, reports, and presentation materials

INDIVIDUAL CONTRIBUTIONS TO THE WORK

Batch Member 3 : DEEPAK KUMAR V (7376222CT107)

1. Implemented server-side logic for core modules such as job posting, proposals, and contracts
2. Designed and managed database schemas and relationships
3. Developed authentication, authorization, and role-based access control
4. Implemented escrow simulation, payment flows, and ledger tracking

PENDING WORKS AND PLAN FOR COMPLETION

Pending Works

1. Real-Time Chat System
2. Video Call Integration
3. Milestone-Based Escrow System
4. Device fingerprinting for session tracking and fraud prevention
5. Dispute Resolution Workflow

Plan for Completion

Phase 1: Finalize real-time chat system and message persistence

Phase 2: Integrate video call functionality with role-based access

Phase 3: Upgrade escrow system to milestone-based payments

Phase 4: Enhance security features and audit mechanisms

Phase 5: Perform comprehensive testing, optimization, and documentation

PLAN FOR PUBLICATIONS

- Journal / Conference Identified for Submission
- Tentative Dates for Submission / Presentation / Acceptance

Status on Partial Completion and Submission of Project Report

SL.No	List of Documents	Status (Provide the drive link of prepared document)
1	Cover Page & Title Page	COMPLETED : Cover Page.pdf
2	Bonafide Certificate	COMPLETED : Bonafide.pdf
3	Declaration	COMPLETED : Declaration.pdf
4	Acknowledgement	COMPLETED : Acknowledge.pdf
5	Chapter I – Introduction	COMPLETED : Introduction.pdf
6	Chapter 2 – Literature Survey	COMPLETED : Literature Survey.pdf

REFERENCES
(Journal Papers/ Books/ Website References)

[1]. I. Sommerville, *Software Engineering*, 10th ed., Pearson Education, 2016.

[2]. M. Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley, 2003.

[3]. S. Tilkov and S. Vinoski, “Node.js: Using JavaScript to Build High-Performance Network Programs,” *IEEE Internet Computing*, vol. 14, no. 6, pp. 80–83, Nov.–Dec. 2010.

[4]. R. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” Ph.D. dissertation, Univ. of California, Irvine, 2000.

[5]. OpenJS Foundation, “Node.js Documentation,” https://nodejs.org, accessed Jan. 2026.

[6]. Prisma Labs, “Prisma ORM Documentation,” https://www.prisma.io, accessed Jan. 2026.

[7]. NextAuth.js Team, “NextAuth.js Authentication Documentation,” https://authjs.dev, accessed Jan. 2026.

[8]. PostgreSQL Global Development Group, “PostgreSQL Documentation,” https://www.postgresql.org/docs, accessed Jan. 2026.